

# LA CAPA DE TRANSPORTE

- Permite independizarse de la subred de abajo. (de la capa inferior)

## El servicio de transporte

Servicios proporcionados a las capas de superiores:

- Proporcionar un servicio **eficiente, confiable** (control de flujo y errores) y **económico** a sus usuarios.
- Existen dos servicios:
  - Orientado a la conexión
  - No Orientado a la conexión.

MUY SEMEJANTE A CAPA DE RED

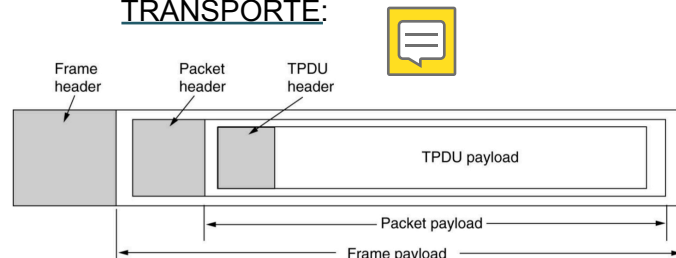
## ¿POR QUE NO ES SUFICIENTE UNA SOLA CAPA?

- nadie tiene el control de la subred de comunicaciones → los hosts no tienen control
- la capa de transporte sí se ejecuta a nivel de host, lo sí permite controlar un poco lo que sucede a la capa de red

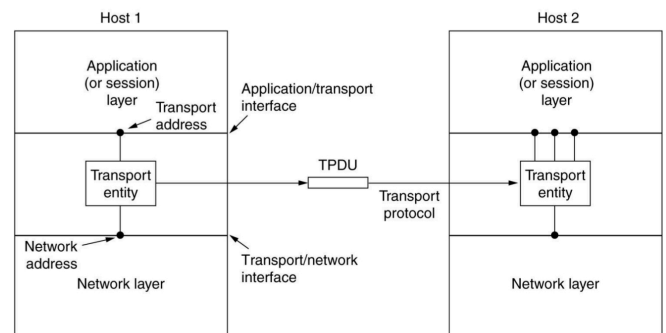
## Entidad de Transporte

- arma la **TPDU** para poder mandarla entre hosts

**TPDU = UNIDAD DE DATOS DEL PROTOCOLO DE TRANSPORTE:**



cabecera (información de control) + datos (payload)



Capa 1 a 4 - proveedor del servicio de transporte  
Capa 5 a 7 - usuarios del servicio de transporte

## Primitivas del servicio de Transporte

(operaciones básicas que se pueden hacer)

| Primitiva         | Paquete enviado     | Significado                                                                          |
|-------------------|---------------------|--------------------------------------------------------------------------------------|
| <u>LISTEN</u>     | Ninguno             | Se bloquea hasta que algún proceso intenta la conexión                               |
| <u>CONNECT</u>    | CONNECTION REQ      | Intenta activamente establecer una conexión<br><i>↳ de acuerdo a la rta: - - - -</i> |
| <u>SEND</u>       | DATA                | Envía información                                                                    |
| <u>RECEIVE</u>    | Ninguno             | Se bloquea hasta que llega un paquete DATA                                           |
| <u>DISCONNECT</u> | DISCONNECTI ON REQ. | Este lado quiere liberar la conexión.                                                |

El host se queda esperando con la primitiva **LISTEN** sin enviar ningún tipo de paquete hasta que algún otro host trate de iniciar una conexión. Un host puede tratar de iniciar una conexión a partir de la primitiva **CONNECT** lo cual requiere de un paquete CONNECTION REQUEST. Si es aceptada la conexión puede entonces comenzar a enviar información a través de una primitiva **SEND** con paquetes tipo DATA. Si el host en esta conexión espera recibir un dato entonces utiliza la primitiva **RECEIVE** sin enviar ningún paquete. Y por último si desea liberar la conexión debe utilizarse una primitiva **DISCONNECT** enviando un paquete DISCONNECTION REQ.

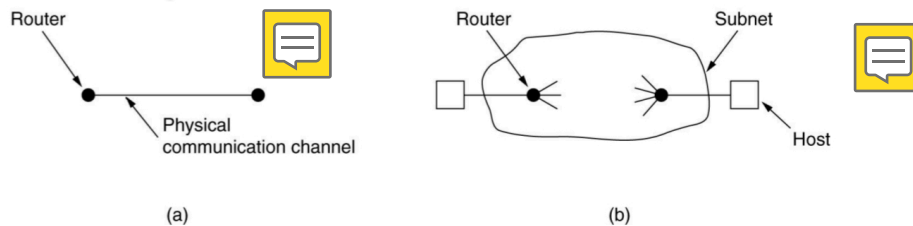
## Proceso en capa de transporte

(ejemplo de cómo se ejecutarían las primitivas)

- El servicio ejecuta una primitiva LISTEN
- Un cliente quiere conectarse a un servidor y ejecuta una primitiva CONNECT.
- El CONNECT invoca a una TPDU CONNECTION REQUEST al servidor.
- El servidor envía una TPDU CONNECTION ACCEPTED.
- Ahora los datos se envían usando las primitivas SEND y RECEIVE.
- Cuando ya no se necesita una conexión se emite un TPDU DISCONNECT.

## Elementos de los protocolos de Transporte

(comparación enlace punto a punto vs transporte)



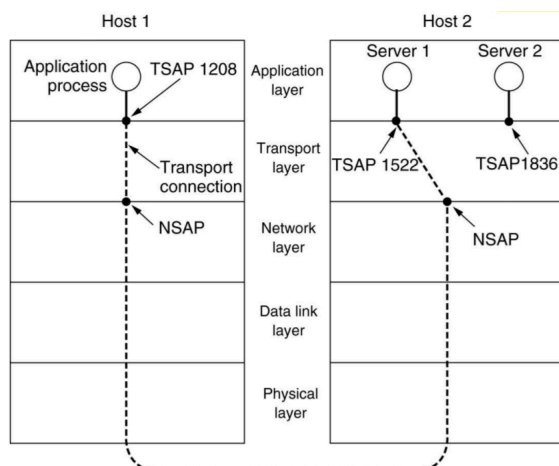
| Capa de enlace <i>ejemplo punto a punto</i>                 | Capa de transporte                                                                               |
|-------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| No es necesario direccionamiento                            | Se requiere saber la dirección destino                                                           |
| El establecimiento de la conexión es sencilla.              | El establecimiento de la conexión es mas compleja.                                               |
| No existe capacidad de almacenamiento entre los enrutadores | Existe la posibilidad que los paquetes queden almacenados en la subred durante un tiempo finito. |
| Se necesitan buffer                                         | Se necesitan buffer pero de mayor capacidad                                                      |

## Direccionamiento

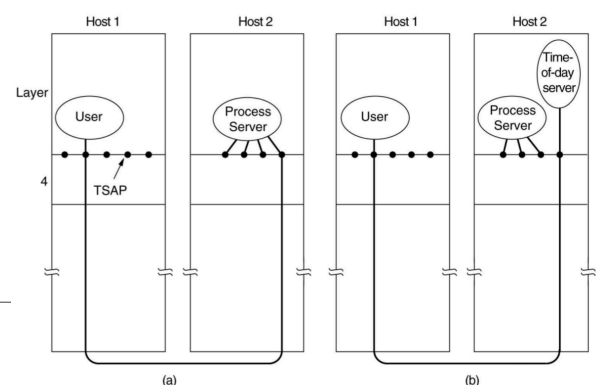
El método que normalmente se emplea es definir direcciones de transporte en las que los procesos pueden estar a la escucha de solicitudes de conexión. En internet se llaman **puertos**. → (dir. 16 bits)



- Hay rangos de puertos predefinidos a ciertas aplicaciones con mucha difusión, sin embargo uno como host puede configurar eso y usar cualquiera
- Para conectarme a una aplicación necesito sí o sí un puerto



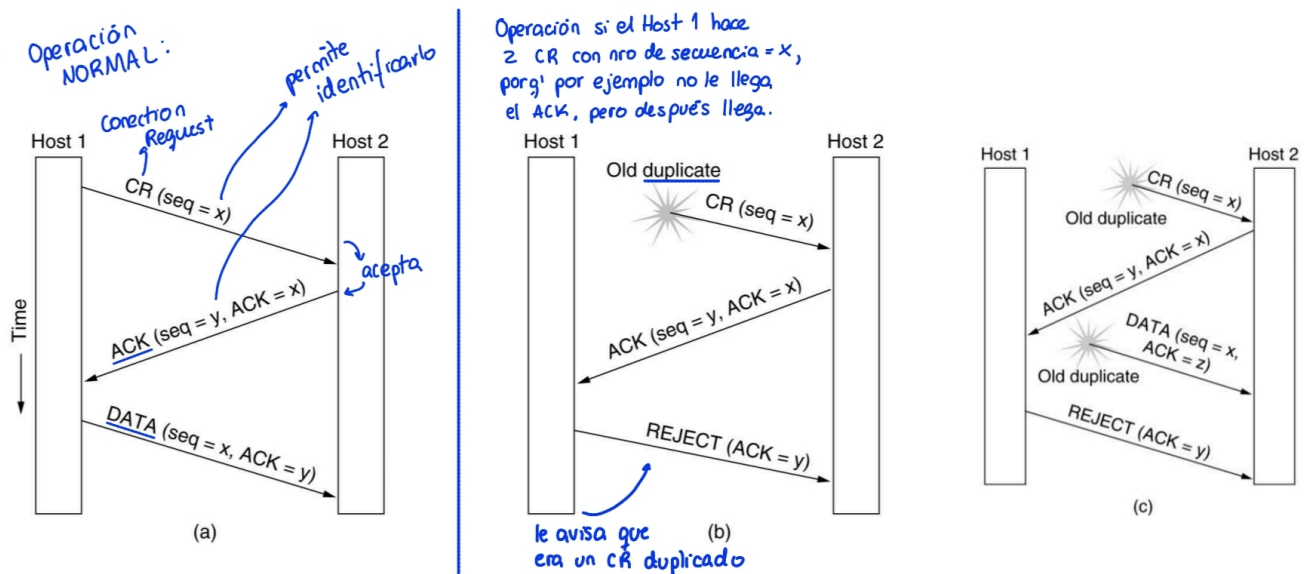
TSAP: Punto de acceso al servicio de Transporte  
NSAP: Punto de acceso al servicio de RED



## Establecimiento de una conexión (de TCP)

Desarrollo del **acuerdo de tres vías**.

→ Es el método que usa el protocolo **TCP (Transmission Control Protocol)** para establecer una **conexión confiable** entre dos dispositivos antes de que puedan empezar a intercambiar datos.



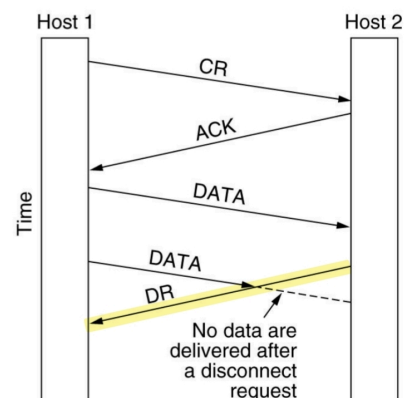
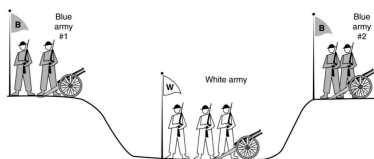
- a) Operación normal
- b) CONNECTION REQUEST duplicadas
- c) CONNECTION REQUEST duplicadas y ACK duplicada

## Libерación de una conexión

Puede ser simétrica o asimétrica:

- **Asimétrica:**
  - Más abrupta y puede provocar pérdida de datos
  - Es problemático si tengo mucho volumen de datos o información sensible
- **Simétrica:**

• Problema de los dos ejércitos

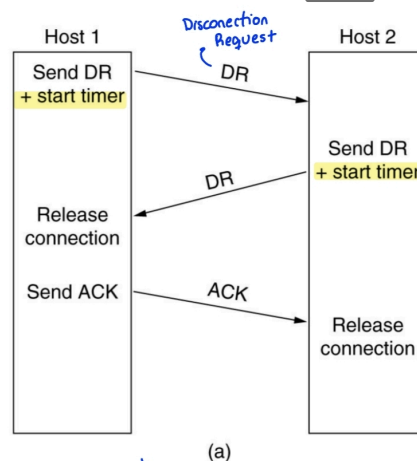


**Asimétrica**

¿Cómo hacer una simétrica?

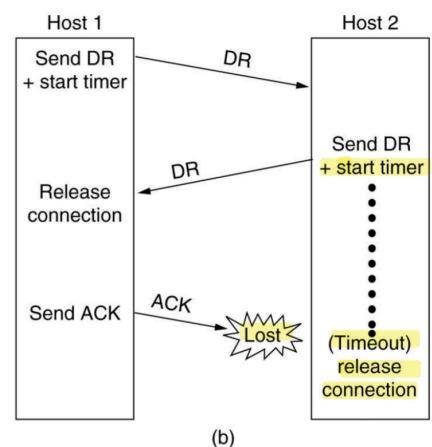
→ con temporizadores

**Caso Normal:**

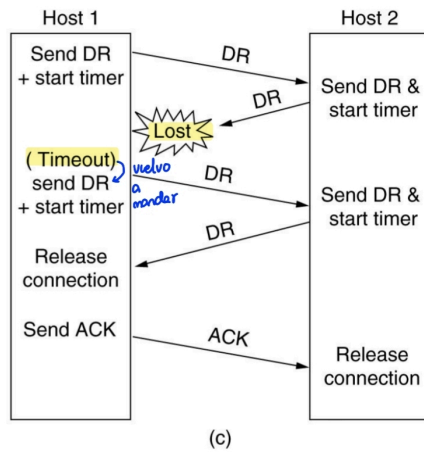


*una especie de ac. de tres vías*

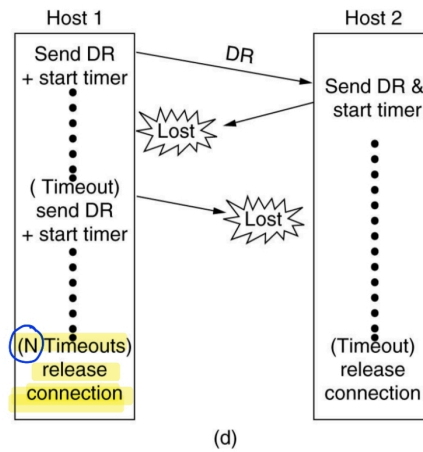
**Problema 1**



### Problema 2:



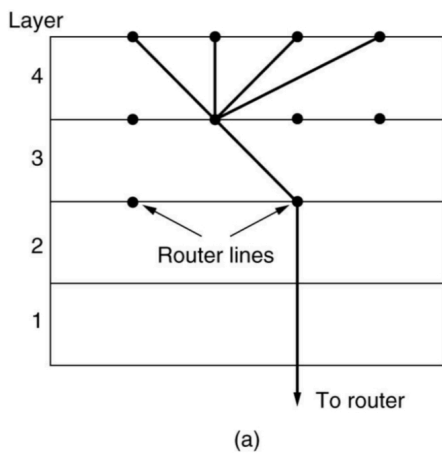
### Problema 3:



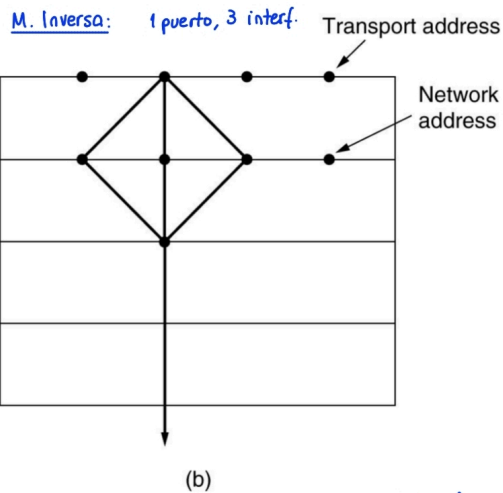
## Multiplexión

Un host con una dirección de red, se conecta a muchas direcciones de transporte, muchas aplicaciones.

### M. Común:



### M. Inversa:



### ¿Cómo se hace?

→ [dir ip] : [puerto] → socket ∈ [0,65535] → podríamos tener esa cantidad de direcciones simultáneas

## Protocolos de Transporte de Internet

Internet tiene dos protocolos en la capa de transporte:

- ★ **UDP** (Protocolo de Datagrama de Usuario) → no confiable
- ★ **TCP** (Protocolo de Control de Transmisión) → confiable

### Cabecera IP.

### Especificación del Protocolo de Transporte:

|                       |           |                              |      |          |                |                         |  |
|-----------------------|-----------|------------------------------|------|----------|----------------|-------------------------|--|
| ←————— 32 bits —————→ |           |                              |      |          |                |                         |  |
|                       |           |                              |      |          |                |                         |  |
| Versión               | Lon. Cab. | DS (DiffServ)                |      |          | Longitud Total |                         |  |
| Identificación        |           |                              | Res. | DF       | MF             | Desplazam. de Fragmento |  |
| Tiempo de vida (TTL)  |           | Protocolo                    |      | Checksum |                |                         |  |
|                       |           | Dirección de origen          |      |          |                |                         |  |
|                       |           | Dirección de destino         |      |          |                |                         |  |
|                       |           | Opciones (de 0 a 40 octetos) |      |          |                |                         |  |

| Valor | Protocolo |
|-------|-----------|
| 1     | ICMP      |
| 4     | IP        |
| 6     | TCP       |
| 17    | UDP       |
| 89    | OSPF      |

Esto son solo algunos ejemplos de los valores que puede tener el campo 'protocolo'

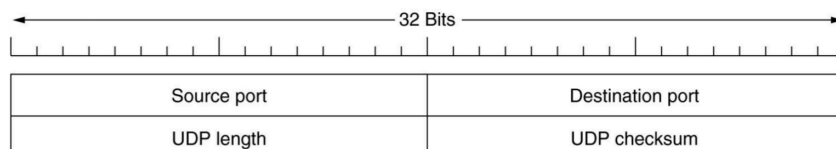


## ★ Protocolo UDP (RFC 768)

- Proporciona una forma para que las aplicaciones envíen datagramas IP encapsulados sin tener que establecer una conexión.
- Servicio **sencillo, pero NO fiable.**
- Transmite **segmentos** que consisten: [ **encabezado de 8 bytes + carga útil** ]
- Multiplexa los datos de las aplicaciones y efectúa opcionalmente una comprobación de errores, pero no hace:
  - Control de Flujo
  - Control de congestión
  - Conexión/Desconexión
  - Retransmisión de datos perdidos.
- Se utiliza en los siguientes casos:
  - El intercambio de mensajes es muy escaso, ej consultas al DNS
  - La aplicación es en tiempo real (+ importante que llegue a tiempo). Ej. Videoconferencias, VoIP.
  - Los mensajes se envían regularmente y no importa si se pierde alguno. Ej. SNMP.
  - Se envía tráfico broadcast/multicast.
  - Tráfico de P2P o IPTV.

→ Utilizar UDP le da a los paquetes IP la posibilidad de sumar puertos de origen y destino.

### La cabecera UDP



- Puerto Origen
- Puerto Destino
- Longitud UDP: **incluye el encabezado de 8 bytes y los datos.**
- Checksum: **es opcional** y se almacena como 0 si no se calcula.

## ★ Protocolo TCP (RFC 793)

- Proporciona un flujo de bytes **confiable de extremo a extremo.**
- Ofrece un servicio de transporte orientado a la conexión.
- Los paquetes TCP se llaman **segmentos.**
- Los Socket son los puntos terminales del cliente y del servidor.
- Cada **byte** de una conexión TCP tiene su propio **número de secuencia** de 32 bits.
- Un **segmento** consiste en un **encabezado TCP fijo de 20 bytes** (más una parte opcional) seguidos de cero.

- **Dos límites restringen el tamaño de segmento:**

- Cada segmento incluido el encabezado TCP, debe caber en la carga útil de 65515 bytes de IP.
- Cada red tiene una unidad máxima de transferencia (MTU) y cada segmento debe caber en la MTU.

- **Utiliza el protocolo de ventana corrediza.**

- Cuando llega un segmento al destino, la entidad TCP receptora devuelve un segmento que contiene un número de confirmación de recepción igual al siguiente número de secuencia que espera recibir. Si el temporizador del emisor expira antes de la recepción de la confirmación el emisor envía de nuevo el segmento.

## Funciones del TCP

- Los socket pueden utilizarse para múltiples conexiones. **MULTIPLEXACIÓN**
- Usa el acuerdo de tres vías.
- Controlar errores, retransmitiendo segmentos perdidos o erróneos. Elimina duplicados.
- Establecer y terminar conexiones.
- Gestionar los búferes y ejercer control de flujo de forma eficiente.
- Gestionar el intercambio de datos de forma eficiente en la red.
- Efectuar control de congestión.
- Transmisiones Dúplex Total y punto a punto → los 2 extremos pueden enviarse de forma bidireccional datos.

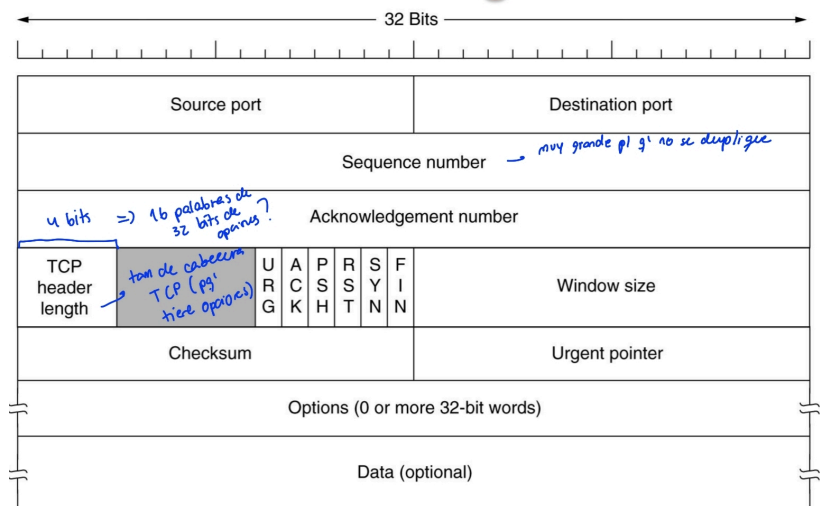
### \* Servicio DNS:

- Como servicio de resolución de direcciones → usa UDP
- Para copiar la base de datos de un servidor DNS a otro → usa TCP (porque necesito confiabilidad)

## Encabezado del segmento TCP

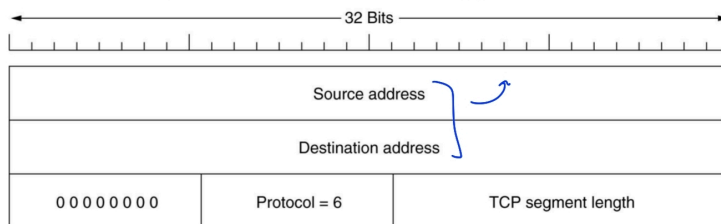
→ Es más complejo que el de UDP

- **Puerto origen y destino:** identifican los puntos terminales locales de la conexión.
- **Número de secuencia y confirmación:** El segundo especifica el siguiente byte esperado, no el último byte correctamente recibido.
- **Longitud del encabezado:** indica la cantidad de palabras de 32 bits contenidas en el encabezado TCP.



- **URG:** apuntador urgente.
- **ACK:** 1 confirmación válida.
- **PSH:** indica datos que se deben transmitir de inmediato.
- **RST:** restablece una conexión.
- **YN:** para establecer una conexión.
- **FIN:** para liberar una conexión.
- **Tamaño de ventana:** indica la cantidad de bytes que pueden enviarse comenzando por el byte cuya recepción se ha confirmado. → esa ventana va a cambiar y va a depender de cómo esté funcionando mi subred de comunicaciones.

→ **Suma de verificación:** agrega confiabilidad. Es una suma de verificación del encabezado, los datos y el *pseudo encabezado conceptual*. → saca datos de la capa de red y calcula ese checksum nuevo.



→ **Opciones:** ofrece una forma de agregar características extras no cubiertas por el encabezado normal. Ej. Carga útil máxima que está dispuesta a aceptar.

## Establecimiento de una conexión TCP

Se utiliza el acuerdo de tres vías.

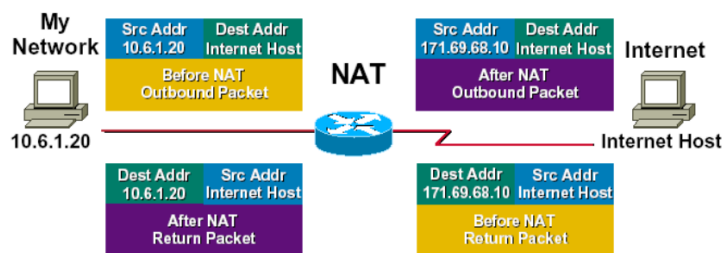
1. El servidor espera pasivamente una conexión entrante ejecutando las primitivas LISTEN y ACCEPT y especificando cierto origen o bien nadie en particular.
2. El cliente ejecuta una primitiva CONNECT especificando la dirección y el puerto IP. Esta primitiva envía un segmento TCP con el bit SYN encendido y el bit ACK apagado.
3. Si el servidor rechaza, envía un segmento con el RST encendido.
4. Si el servidor acepta, envía un segmento confirmación de recepción

**preguntar si hay que agregar lo que sigue**

## NAT Y PAT

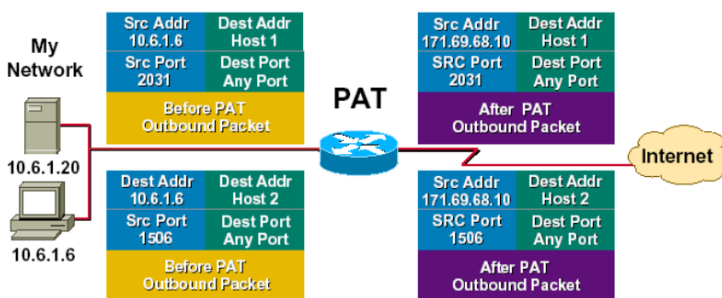
### Concepto básico de NAT (Network Address Translation)

→ La Traducción de Direcciones de Red (NAT) cambia las direcciones IP en el encabezado IP.



### Concepto básico de PAT (Port Address Translation)

→ La Traducción de Direcciones de Puertos (PAT) extiende a NAT de “uno a uno” a “varios a uno” asociando el puerto origen con cada flujo.



### NAT

→ Protocolo utilizado por falta de ip. Traduce una ip a otra.

- **Problema:** El espacio de direccionamiento IP está limitado y el obtener un bloque de direcciones registradas es difícil.
- **Solución:** Utilizar direcciones IP privadas (RFC 1918) en la red interna.
- Las direcciones IP privadas que se pueden utilizar para la red interna son:

Clase A: 10.x.x.x

Clase B rango: 172.16.x.x–172.31.x.x

Clase C rango: 192.168.1.x–192.168.254.x

- **NAT cambia una IP privada** (generalmente la red interna) **por una IP pública** (la del router o una asignada a él) **cuando el tráfico sale hacia internet** (podría ser de privada a privada, o pública a pública, pero el uso más común es el de privada a pública).
- Este **cambio es a nivel de red** ya que estás modificando la dirección IP.
- El **checksum** de la cabecera IPv4 se invalida al modificar la cabecera IP cambiando la dirección de origen. El router que realiza el NAT **debe recalcularlo y actualizarlo para que el paquete no sea descartado** por los routers intermedios en su camino.
- El router NAT mantiene una **tabla de traducciones** a quién le prestó la ip pública para cuando vuelva el paquete.
- NAT **mejora la seguridad** al ocultar las direcciones IP privadas de la red interna. Desde internet, solo se ve la dirección IP pública del router, lo que dificulta que atacantes identifiquen y ataquen directamente a tus servidores o dispositivos internos.

**Problemas:** Si el router solo tiene una IP pública y dos servidores internos intentan iniciar una conexión con el mismo destino usando la misma IP pública de origen, el router no tendría cómo diferenciarlos al regreso → (para esto se usa PAT). Si 2 sv van a destinos distintos no hay problema, porque la **tabla almacena origen y destino**.

## Beneficios de NAT

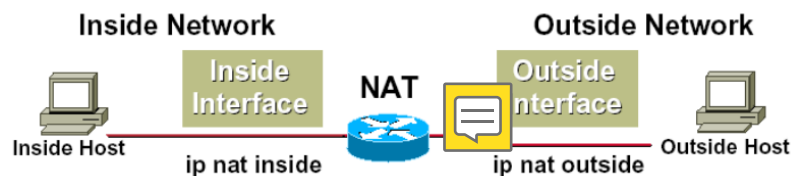
- Reemplaza la dirección de origen con una dirección ruteable permitiendo a hosts con direcciones privadas acceder a internet.
- Provee de conectividad transparente, escalable y bidireccional entre distintas oficinas de la misma empresa.
- Elimina la necesidad de reasignar números a los hosts al cambiar de ISP o de esquemas de direccionamiento.
- Fortalece la privacidad de red puesto que las direcciones asignadas se encuentran ocultas. Evita el escaneo de puertos en la red.

## Disponibilidad y Soporte de Aplicaciones de NAT

- Disponible en las versiones de IOS posteriores a la 11.2. (Octubre de 96).
- Soportado en todas los sistemas basados en IOS. (excepto los Cisco 7000).
- Las aplicaciones son transparentes para el Cisco IOS de NAT, con excepción de que:
  - la aplicación requiera puertos específicos o negociados
  - ó cuente con direcciones IP preestablecidas

## Conceptos y Terminología de NAT

- Una interfaz en el router puede ser definida como **inside** o **outside**.

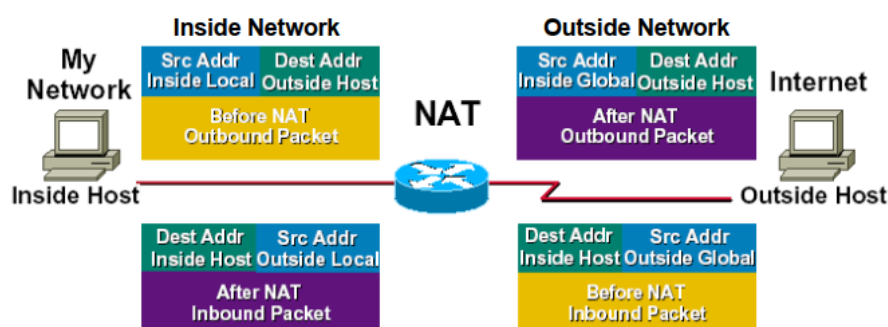


→ Cuando se configura NAT en un router, necesitas decirle al router qué interfaces "pertenecen" a tu red interna (donde están las IPs privadas) y qué interfaces "pertenecen" a la red externa (donde están las IPs públicas, como Internet). Para esto, se designan las interfaces como inside (interna) o outside (externa).

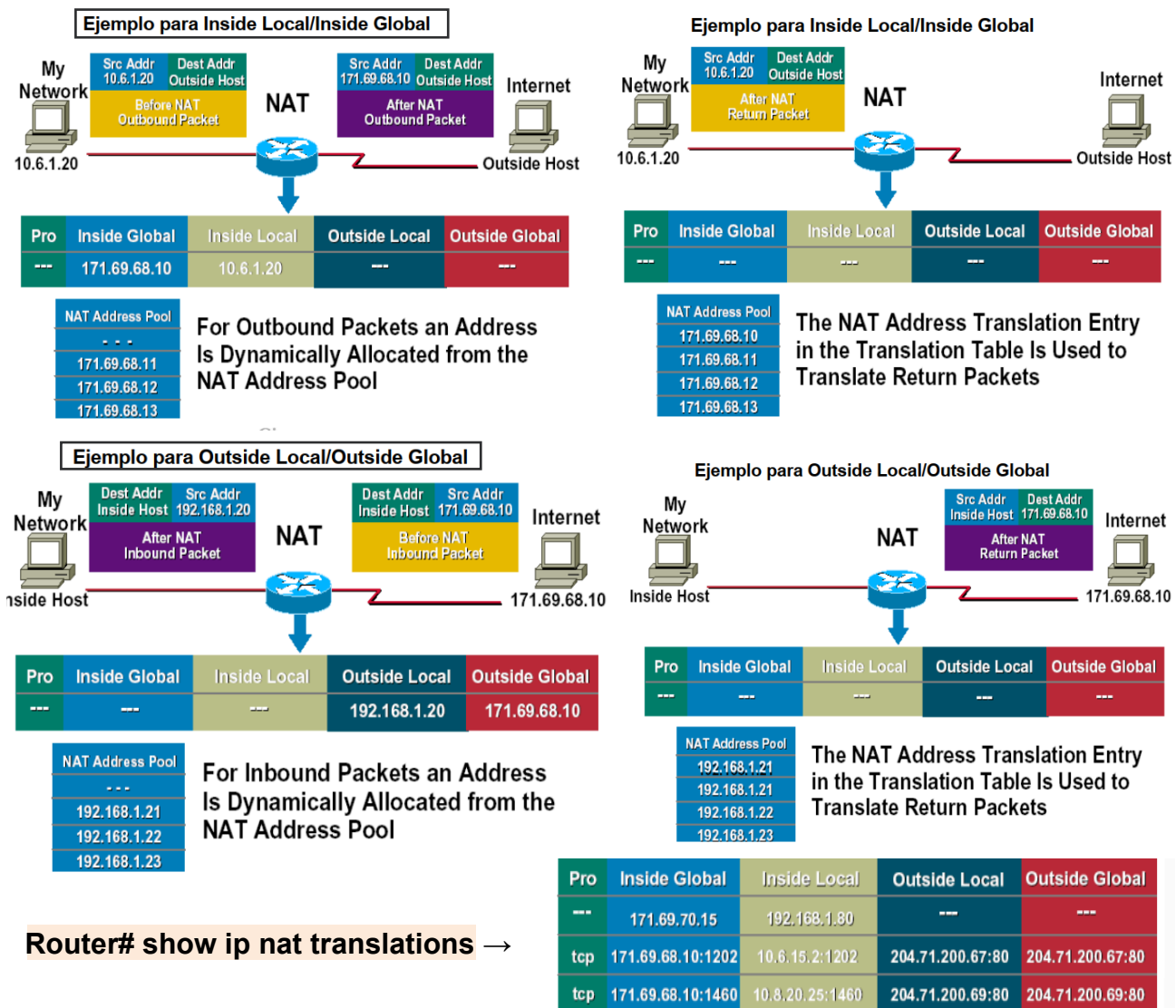
- Las traducciones ocurren sólo desde dentro hacia fuera ó viceversa – nunca entre el mismo tipo de interfaz.
- Las traducciones NAT pueden ser **estáticas** o **dinámicas**.
- **Estáticas:** son directamente introducidas en la configuración y se encuentran siempre en la tabla de traducciones.
  - Se utilizan cuando se desea iniciar conexión desde ambas interfaces (inside y outside). Ej: SMTP, Web.
  - Se utilizan cuando se desea que un host en específico sea traducido a una IP específica.

```
ip nat inside source static 10.6.1.20 171.69.68.10
```
- **Dinámicas:** utilizan listas de acceso para identificar las direcciones IP para las cuales NAT debe crear traducciones.
  - Se utilizan cuando se desea iniciar la conexión sólo desde una interfaz.

```
ip nat inside source list 1 pool nat-pool
access-list 1 permit 10.0.0.0 0.255.255.255
```



- Una dirección IP puede ser **local o global**:
  - Una IP local es vista en la red interna.
  - Una IP global es vista en la red externa.
- **Inside local** → Es una dirección IP asignada a un host de la red interna.
- **Inside global** → Es la manera en la cual se presenta la dirección IP de un host interno a un host y a la red externos.
- **Outside local** → Es la dirección IP de un host externo de la forma que se le presenta a la red interna.
- **Outside global** → Es la dirección IP configurada y asignada a un host en la red externa.



Una traducción de NAT es de 1 a 1 o de varios a 1.

- Las traducciones **1 a 1 (NAT)** asignan un diferente IP a cada traducción.
- Las traducciones **varios a 1 (PAT)** pueden asignar el mismo IP para cada traducción

Una traducción NAT puede ser simple o extendida.

- Ambas, NAT y PAT pueden tener traducciones extendidas, pero sólo NAT puede tener traducciones simples.





- **ip nat inside source**

- Se utiliza cuando se tienen hosts con dir. IP que no deben ser vistos en la red externa.
- Crea una traducción y traduce la dir. IP de origen para los paquetes que van de dentro hacia afuera. inside -> outside
- Traduce la dir. IP destino para paquetes que van de fuera hacia dentro. outside -> inside

- **ip nat outside source**

- Se utiliza cuando se están utilizando los mismos IP en ambas redes, interna y externa, (redes que se traslapan) ó cuando se tienen varios gateways. Generalmente se utiliza junto con inside source.
- Crea una traducción y traduce la dir. IP de origen para los paquetes que van de afuera hacia adentro. outside -> inside
- Traduce la dir. IP destino para paquetes que van de dentro hacia afuera. inside -> outside

## PAT

→ PAT es para poder tener varias ip a un mismo destino.

→ Tiene una tabla que anota además el puerto por el que vino el mensaje:

| IP destino | Puerto src | IP src | Puerto dst |  |
|------------|------------|--------|------------|--|
|------------|------------|--------|------------|--|

- PAT es una forma de NAT que no solo cambia la IP de origen, sino que también modifica el **número de puerto de origen** en la cabecera del segmento TCP o UDP.
- *Permite traducción de varios a uno.*
- Mapea múltiples dir. IP a una o a unas cuantas → **Una IP pública, muchos dispositivos:** PAT permite que múltiples dispositivos en tu red privada compartan una **única IP pública** para acceder a internet.
- **Un número único de puerto origen identifica cada sesión.**
- También se le llama NAPT en los documentos de la IETF.
- Utiliza un pool de dir. IP
- Puede utilizar un nombre de interfaz.
- Las dir. IP para las interfases pueden ser asignadas:
  - 1) Configurado estáticamente.
  - 2) Vía PPP
  - 3) Vía DHCP en interfaces Ethernet [12.1(2)T].

## NAT vs PAT

- **NAT** → Cuando hay suficiente número de dir. IP para traducciones 1 a 1.
- **PAT** → Cuando hay un número insuficiente de dir. IP disponibles para traducir todos las dir. Internas (inside addresses).

**¿Cómo entro?** Hasta ahora vimos cómo salir de la red con NAT/PAT, pero ahora cómo escucho cosas de afuera estando en la red privada? ip forwarding: **dejo un puerto en escucha.** Configuras una regla en el router para que, cuando el tráfico llegue a su IP pública por un puerto específico ese tráfico sea redirigido a la IP privada y puerto de tu servidor o dispositivo dentro de tu red.

Un servicio por puerto público: Es cierto que solo puedes tener un servidor escuchando en el mismo puerto público para el mismo protocolo (TCP/UDP) en tu IP pública.

# PROTOCOLOS DE CAPA DE APLICACIÓN

## DNS (Domain Name System)

Su finalidad es facilitar el manejo de direcciones IP

www.fich.unl.edu.ar es equivalente a 190.122.240.31

El sistema de nombres de dominio se basa en un esquema jerárquico que permite asignar nombres, basándose en el concepto de dominio, utilizando para su gestión una **base de datos (BBDD) distribuida**.

Las **consultas al DNS** son realizadas por los clientes a través de las rutinas de resolución ("resolver" o resolvedor o resolutor). → es una query al **servidor DNS** (el resolutor).

- Es por ello que no hay un solo servidor DNS.

– Estas funciones son llamadas en cada host desde las aplicaciones de red (ping, telnet, ssh, ...)

→ si no funciona DNS, no podemos acceder a nada, porque implicaría recordar las direcciones IP de cada servicio a acceder.

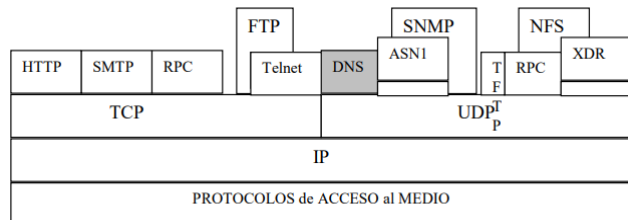
## Cliente/servidor DNS

- Los servidores DNS contienen información de un segmento de la BBDD distribuida y la ponen a disposición de los clientes.

- **Las peticiones de los clientes viajan en paquetes UDP al DNS local (puerto 53).**

(funciona sobre UDP, puerto 53)

– **TCP para transferencias de zona.**



→ ¿Por qué con **UDP**? Ya que necesito velocidad, TCP sería muy lento por el acuerdo de 3 vías.

→ ¿Y **TCP** para qué? Lo usa para copiar información de la base de datos de un servidor DNS a otro.

→ Si capturamos con Wireshark luego de hacer un ping con un dominio en lugar de una IP, veremos que los primeros paquetes serán del protocolo DNS. (ver ejemplo en diapos)

## Ventajas del DNS

- Desaparece la carga excesiva en la red y en los hosts: ahora la información está distribuida por toda la red, al tratarse de una **BBDD distribuida**.
- No hay Duplicidad de Nombres: el problema se elimina debido a la existencia de dominios controlados por un único administrador. Puede haber nombres iguales pero en dominios diferentes.

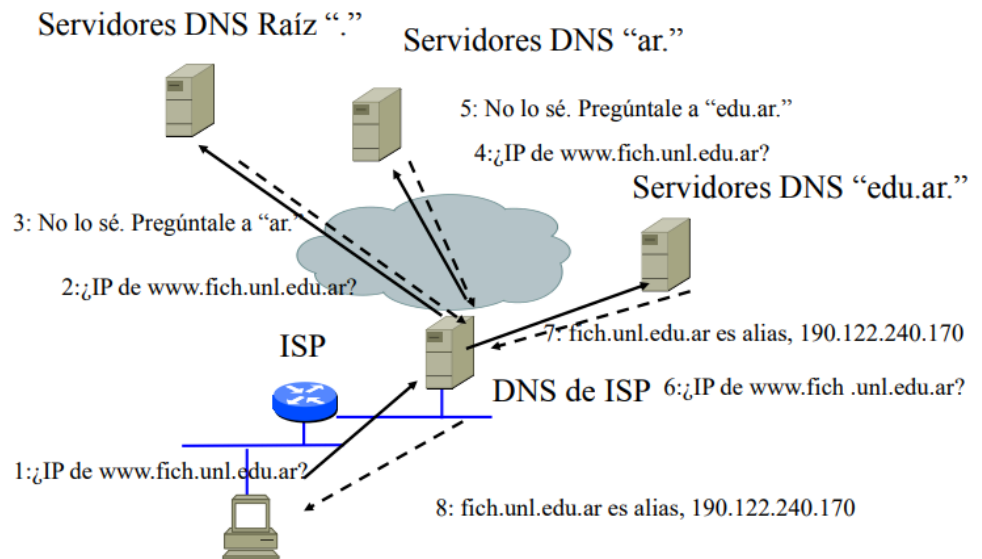
- Consistencia de la Información: ahora la información que está distribuida es actualizada automáticamente sin intervención de ningún administrador.

## Funcionamiento del DNS

Paso 1: Consulta al servidor DNS configurado dentro del host.

Paso 2: Si ese servidor no lo tiene, lo va a buscar a un nivel de jerarquía más arriba.

Paso 3: Si todavía no lo sabe, va al servidor raíz



`fich` es un subdominio de [unl.edu.ar](http://unl.edu.ar), => después podría ir a ese para que me diga cuál es el `fich`

pero puede ser que no haga falta ir hasta el raíz.

## Elementos del DNS

1. La sintaxis del nombre

2. La implementación de la base de datos

Comandos y ficheros relacionados con el DNS

### Sintaxis del nombre (1/3): definición

**Nombre de dominio** es una cadena de hasta 255 caracteres, formada por etiquetas separadas por puntos (cada etiqueta inferior a 64 caracteres) de forma jerárquica o por niveles (comenzando el nivel superior por la derecha).

→ Cada dominio es un índice en la BBDD del DNS.

→ No se distinguen mayúsculas de minúsculas. Esto no se aplica a la parte izquierda de @ en las direcciones de correo.

Ejemplo: `fich.unl.edu.ar` tiene 4 etiquetas, siendo el dominio de nivel superior "ar.", dominio de 2º nivel "edu.ar." y dominio de nivel inferior "fich.unl.edu.ar."

Además, de un nombre de dominio puede representar un host.

### Sintaxis del nombre (2/3): absoluto y relativo

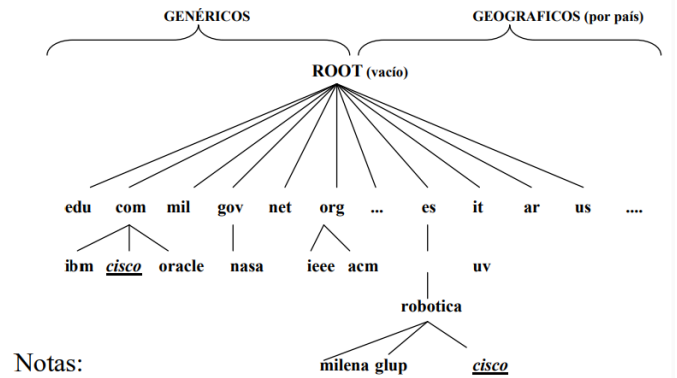
Los nombres de dominio absolutos terminan con "." (ej. "`fich.edu.`") y los relativos no. Se necesita saber el contexto del dominio superior para determinar de manera única su significado verdadero.

## Sintaxis del nombre (3/3): clasificación de los dominios

En el nivel absoluto superior o raíz, los dominios se clasifican en:

- **Geográficos** – división por países (o regiones)
- **Genéricos** – en función del tipo de organización

→ Dentro del root hay dominios considerados como genéricos y otros son geográficos, donde cada país los vende como quiere, y además administra cuales son los dominios de segundo nivel que se pueden otorgar en el mismo.



Notas:

- (1) Cada dominio absoluto se define desde la hoja del árbol hasta la raíz
- (2) Puede haber nombres duplicados en dominios diferentes (ej “cisco”)

## Registro de Recursos (RR) (1/3)

Cada entrada en la tabla de un DNS contiene *información*, no sólo de las direcciones IP, si no de un registro de recursos, con 5 campos o tuplas:

|                         |              |                |                                  |
|-------------------------|--------------|----------------|----------------------------------|
| <b>[Nombre_dominio]</b> | <b>[TTL]</b> | <b>[Clase]</b> | <b>Tipo Dato_Registro(Valor)</b> |
|-------------------------|--------------|----------------|----------------------------------|

Cuando un cliente (a través de un resolver) pregunta por un nombre de dominio al DNS, lo que recibe son los RR asociados a ese nombre y por tanto *la función real del DNS es relacionar los dominios de nombres con los RR* Normalmente existen varios RR por dominio

**Nombre\_dominio:** puede haber más de un registro por dominio. Este campo a veces puede omitirse, tomando por defecto el último nombre de dominio indicado con anterioridad.

**TTL:** tiempo de vida. Indicando la estabilidad del registro (tiempo que se guarda en la caché).

- La información altamente estable tiene un valor grande (86400 seg. = 1 día)
- La información volátil recibe un valor pequeño (60 seg.)

→ si yo no le pongo un TTL, me guardaría el dominio de por vida, a pesar de que cambie o lo que sea.

**Clase:** Actualmente sólo se utiliza **IN**, para información de Internet. Este campo si se omite, se toma el último valor indicado con anterioridad

**Dato\_Registro(Valor):** es un número o texto ASCII dependiendo del tipo de registro.

## Tipo de Registro de Recursos (RR) (3/3)

| Tipo de Registro                        | Descripción                                                                                                                                     |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <u>SOA</u><br><i>Start Of Authority</i> | Inicio de autoridad, <u>identificando el dominio o la zona</u> . Fija una serie de parámetros para esta zona.                                   |
| <u>NS</u><br><i>Name Server</i>         | El <u>nombre de dominio</u> se hace corresponder con el nombre de una computadora a de confianza para el dominio o <u>servidor de nombres</u> . |
| <u>A</u><br><i>Address</i>              | Dirección IP de un host en 32 bits. Si este tiene varias direcciones IP, multihomed, habrá un registro diferente por cada una de ellas.         |
| <u>CNAME</u>                            | Es un alias que se corresponde con el nombre canónico verdadero.                                                                                |
| <u>MX</u>                               | Se trata de un intercambiador de correo (Mail eXchanger), es decir, un dominio dispuesto a aceptar solo correo electrónico.                     |
| <u>TXT</u>                              | Texto, es una forma de añadir comentarios a la Base de Datos. Por Ej., para dar la dirección postal del dominio.                                |
| <u>PTR</u>                              | Apuntador, hace corresponder una dirección IP con el nombre de un sistema. Usado en archivos dirección -nombre, la inversa del tipo A.          |
| <u>HINFO</u>                            | Información del Host, tipo y modelo de computadora y SO                                                                                         |
| <u>WKS</u>                              | Servicios públicos (Well -Known Services). Puede listar los servicios de las aplicaciones disponibles en el ordenador.                          |



CNAME → un nombre apuntando a otro nombre:

```
filippag@7cf34de86ed9 ~ % dig www.fich.unl.edu.ar

;<>> DiG 9.10.6 <>> www.fich.unl.edu.ar
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 17873
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;www.fich.unl.edu.ar.      IN      A

;; ANSWER SECTION:
www.fich.unl.edu.ar.      3103    IN      CNAME   fich.unl.edu.ar.
fich.unl.edu.ar.          3103    IN      A        190.122.241.31

;; Query time: 19 msec
;; SERVER: 10.122.1.1#53(10.122.1.1)
;; WHEN: Mon Jun 02 19:03:37 -03 2025
;; MSG SIZE rcvd: 67
```

## Registros MX

**Mail Exchanger:** son servidores de correo ordenados por prioridad en un dominio y registrados en el DNS, de forma que en caso de fallo del principal, generalmente el que tendrá información de todas las cuentas de correo de los usuarios, el cliente de correo (quien quiere realizar la entrega) averiguará a través del DNS el MX del dominio, quien recibirá el correo en nombre del principal.

Este MX intermediario, no requiere tener configuradas las cuentas de correo y en el momento que el principal se reponga, el MX hará entrega de los correos.

## Tipo de preguntas formuladas por los clientes DNS

En el proceso de interrogación, las preguntas pueden ser:

- **Recursiva:** obliga al servidor DNS a que responda aunque tenga que consultar a otros servidores. Esta opción es más frecuente.
- **Iterativa:** el servidor contesta si tiene la información y si no, le remite la dirección de otro servidor capaz de resolver. De esta forma el cliente tiene mayor control sobre el proceso de búsqueda. Esta opción es menos frecuente.
- **Inversa:** permite, dada una IP, consultar el nombre. Para ello se ha creado un dominio especial llamada "in-addr.arpa"

## Soporte para IPv6

- RFC3596 define:
  - Un nuevo tipo de RR (**AAAA**) para la correspondencia de nombre de dominio a dirección IPv6
  - Un dominio para consultas inversas IP6.ARPA
- La versión IP utilizada para la consulta es independiente de la versión de protocolo de los RRs
  - cinco servidores raíz ya tienen dirección IPv6 asignada

otro protocolo de capa de aplicación.....

## DHCP y BOOTP

antes de DHCP:

### BOOTP (BOOT strap Protocol)

→ cuando levantaba pedía la dirección para poderse configurar.

- Corre sobre UDP
- Bit de no fragmentación activado
- Mensajes con longitud fija.
- Archivo de configuración con datos estáticos.
- El sucesor a esta tecnología es el DHCP



### DHCP (Dynamic Host Configuration Protocol)



→ asignación de forma automática y dinámica de direcciones IP.

- La computadora adquiere toda la información en un solo mensaje.
- No se necesita conocer nada de la computadora que solicita la info.
- Tres tipos de asignación:
  - ┆ Asignación manual (BOOTP)
  - ┆ Asignación automática
  - ┆ **Asignación dinámica**
- Utiliza agentes de entrega (Relay Agent)

### Asignación Dinámica

- Asigna direcciones por un tiempo finito.
- Eficiente manejo de un pool de direcciones
- Se utiliza en los siguientes casos:
  - ┆ Pocas direcciones a asignar.
  - ┆ Computadoras con conexiones temporarias.



- **Mecanismo de asignación**

El cliente solicita uso de una dirección por un periodo de tiempo (llamado lease)

El servidor asigna una dirección y la marca como usada

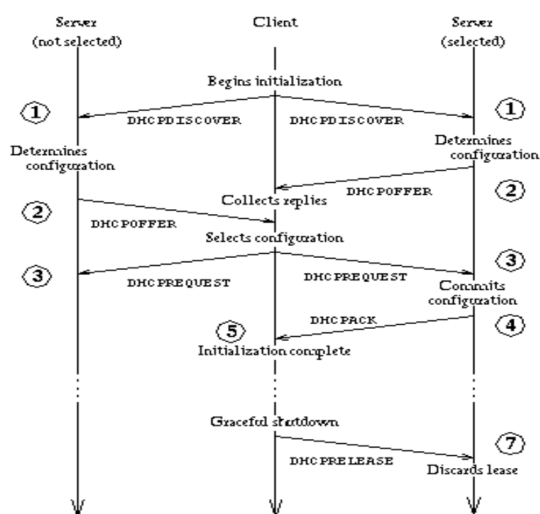
El servidor notifica al cliente de la asignación.

El cliente puede:

- Extender el tiempo
- Preguntar por una dirección permanente
- Devolverla antes de tiempo

## Modelo cliente-servidor

Tenés un cliente y puede haber 2 servers de DHCP en la misma LAN:



- El cliente envía un broadcast DHCPDISCOVER
- Cada Server responde con un mensaje DHCPOFFER
- El cliente recibe todos los DHCPOFFER y elige uno de ellos enviando un DHCPREQUEST
- Los server no elegidos reciben el DHCPREQUEST y liberan esa dirección y el server elegido contesta con un DHCPACK que contiene la configuración de la estación.
- El cliente recibe el DHCPACK y queda configurado.
- Si el cliente recibe un DHCPNACK, comienza nuevamente.
- El cliente puede enviar un DHCPRELEASE. (libera la IP).

## Formato del mensaje DHCP

Los campos más utilizados:

- htype: tipo de dirección de hardware
- hlen: longitud de dirección de hardware
- hops: saltos hasta un DHCP server
- ciaddr: dirección del cliente si la conoce.
- yiaddr: dirección del cliente, entregada por el Server
- giaddr: dirección IP del relay agent
- options: parámetros opcionales (DNS, Gateway, Mascara, etc.)

|               |           |           |          |
|---------------|-----------|-----------|----------|
| op (1)        | htype (1) | hlen (1)  | hops (1) |
| xid (4)       |           |           |          |
| secs (2)      |           | flags (2) |          |
| ciaddr (4)    |           |           |          |
| yiaddr (4)    |           |           |          |
| siaddr (4)    |           |           |          |
| giaddr (4)    |           |           |          |
| chaddr (16)   |           |           |          |
| sname (64)    |           |           |          |
| file (128)    |           |           |          |
| options (312) |           |           |          |

## Ventajas del DHCP

- Simplifica la tarea de asignar direcciones IP en una red de muchas computadoras.
- Hace más fácil sumar, eliminar o mover PC
- Se puede asignar default: Gateway, DNS, WINs, etc.
- Posibilidad de tener muchos hosts con pocas direcciones IP. → Preguntar al profe

## Desventajas del DHCP

- Si el server de DHCP no funciona, toda la red no funciona.
- Difícil de saber las direcciones IP de cada máquina.

## Otro protocolo de aplicación...

### WWW

- Acceso a documentos multimedia vinculados (hipervínculos) distribuidos en Internet

- Modelo cliente/servidor

- **En el cliente:**

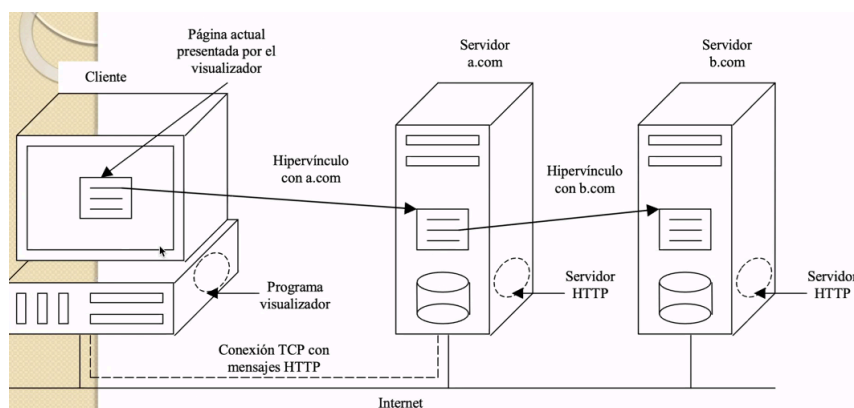
- navegador o browser para mostrar información recibida
- establece una conexión telnet al puerto 80 del servidor y envía comandos para recuperar información → es plano, no está encriptado

- **En el servidor:**

- **proceso TCP** de escucha en puerto 80 → pq estoy haciendo transf de datos.
- Una vez recibida una solicitud de un cliente se envía respuesta y se libera conexión

- **Solicitudes y respuestas según protocolo HTTP (HyperTextTransferProtocol)**

- solicitudes: ASCII
- respuestas: mensajes estructurados tipo MIME



## Telnet

→ para acceder de forma remota a un servidor y usar la consola

- **Terminal Networking**

- Protocolo de acceso a un ordenador desde otro ordenador sobre TCP

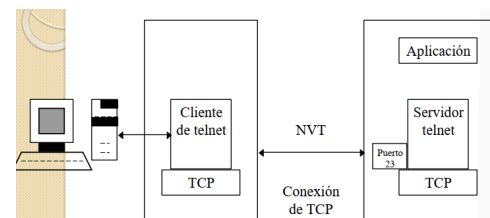
- Disponible para muchos SO incluidos Linux y Windows

- Acceso a través del puerto 23

- Aunque se ideó para comunicaciones peer-to-peer se usa para aplicaciones **cliente/servidor**

- Por **problemas de seguridad** está siendo **sustituido por el ssh**

- Es plano, si pongo una credencial la estoy viendo



# SEGURIDAD EN REDES

## Seguridad en la comunicación

Políticas y prácticas adoptadas para prevenir y supervisar el acceso no autorizado, el uso indebido, la modificación o la denegación de una red informática y sus recursos accesibles.

### Tipos de ataques

- Externos
- Internos
  - Intencionados
  - No intencionados



## Seguridad externa

Nos enfocamos en la seguridad externa: **Firewalls**, **NAT/PAT**, **Control de accesos**, **Cifrado**

**Proxys**,

### Tipos de atacantes

- **Hacker**: tiene un conocimiento profundo de un área y aprovecha este conocimiento a su favor o en favor de otros.
- **White Hat**: hackers éticos. Ponen en conocimiento de las compañías desarrolladoras o empresas las vulnerabilidades, con el objetivo de poder estudiar y corregir los fallos.
- **Black Hat**: criminales causantes de los ciberataques que utilizan todo su conocimiento para introducirse en los sistemas, buscando su beneficio personal/económico.
- **Grey Hat**: hackers que rompen los niveles de seguridad de los sistemas de una empresa para, posteriormente, ofrecer sus servicios para mejorar dicha seguridad.

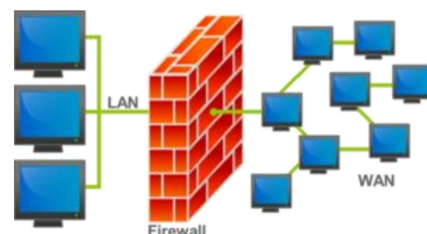
## Firewall o cortafuego

Dispositivo formado por uno o varios equipos que **se sitúan entre la red de la empresa y la red exterior** (normalmente Internet), que **analiza todos los paquetes que transitan entre ambas redes y filtra los que no deben ser reenviados**, de acuerdo con un criterio establecido de antemano.

→ Concentras todo el tráfico en un punto.

### Características:

- Crea un perímetro de defensa y seguridad alrededor de la red a proteger.
- Para que no se convierta en **cuello de botella** en la red, deben de **procesar los paquetes a una velocidad igual o superior al router**.
- Se debe configurar de acuerdo a los servicios que brinde la red, así como conexiones entrantes a la red.
- Realiza **filtrado de paquetes**:
  - **A nivel de red**: direcciones IP
  - **A nivel de transporte**: puertos y tipo de accesos.



- A nivel de aplicación: datos a través de pasarelas para las aplicaciones permitidas analizando el contenido de los paquetes y los protocolos de aplicación (ejemplo servidor proxy).

→ **Violan el esquema de modelo de capas**

## Algunas configuraciones de firewalls

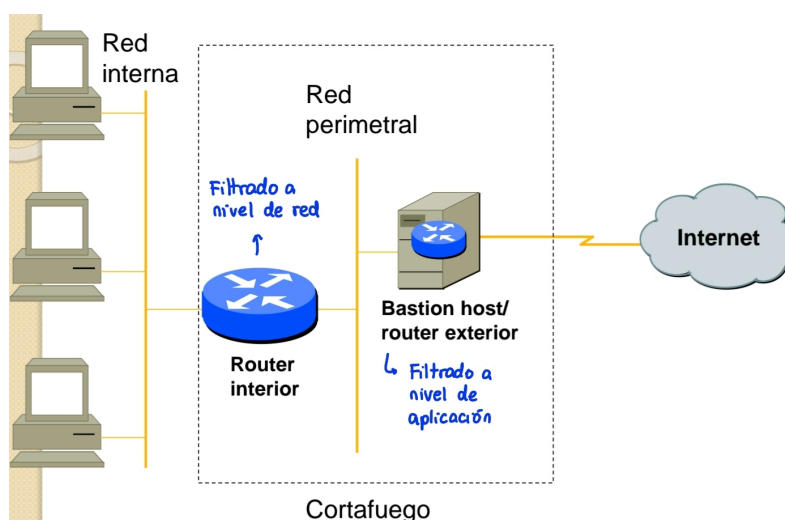
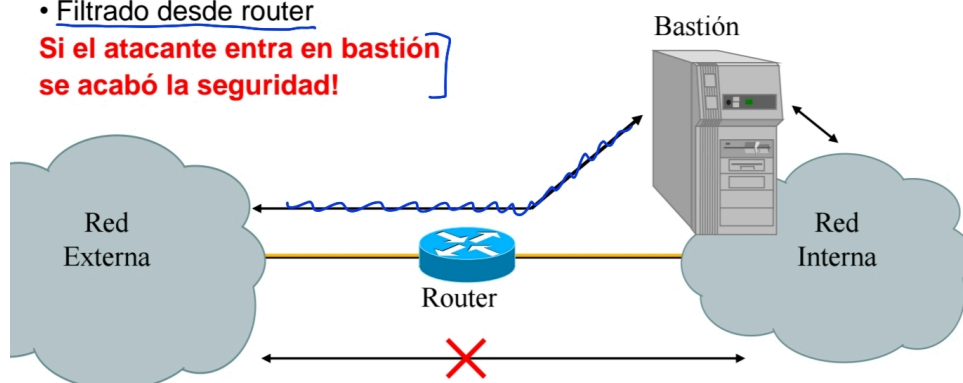
1. **Screened Host Firewall**• un router separando la red interna de Internet, que puede enviar el tráfico de entrada sólo al host bastión.
2. **Dual Homed Gateway**• un host bastión o pasarela para las aplicaciones permitidas separando la red Intranet de Internet
3. **Screened Subnet(DMZ)**• con dos routers separando la red Intranet e Internet y con el host bastión dentro de la red formada por ambos routers. La red interna es conocida como zona neutra de seguridad o zona desmilitarizada.

### 1. Screened Host Firewall

Esta arquitectura permite mantener la conectividad transparente cuando esté justificado, obligando a pasar por el 'bastion host' el resto

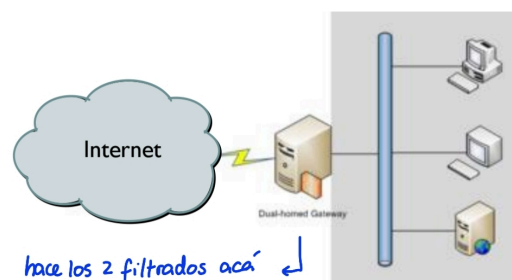
- Router bloquea todo el tráfico hacia la red interna, excepto al bastión
- Servicios vía proxy desde el bastión
- Filtrado desde router

**Si el atacante entra en bastión se acabó la seguridad!**



### 2. Dual-homed gateway

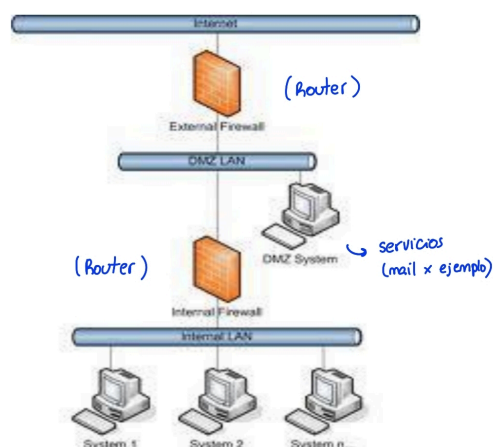
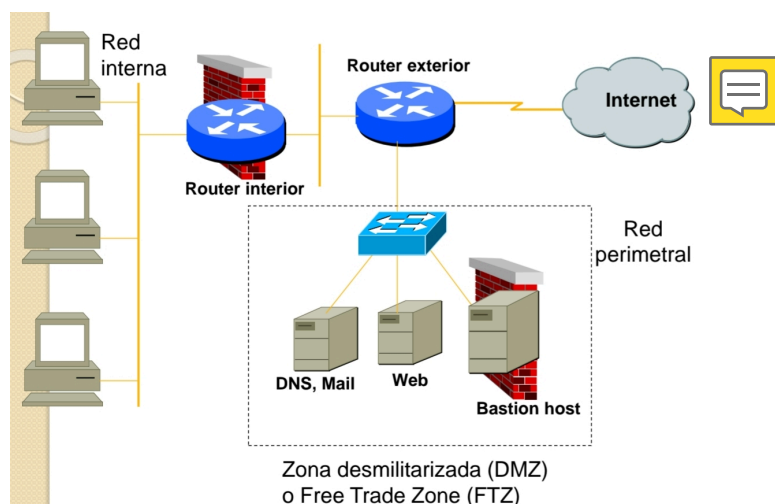
- Un host (bastión) con dos tarjetas de red, conectadas a redes diferentes



- En esta configuración, el bastión puede filtrar hasta capa de aplicación.
- Son sistemas muy baratos y fáciles de implementar
- Sólo soportan servicios mediante proxy El filtrado de paquetes, puede realizarse en Linux a través de “iptables”

### 3. Screened subnet

- Se sitúa una red DMZ (DeMilitarized Zone) entre la interna y la externa, usando dos routers y que contiene el bastión.
- Tráfico sospechoso se envía hacia el bastión, si no puede pasar directamente.
- Soporta servicios mediante proxy (bastión)
- Soporta filtrado de paquetes (routers)
- Implementación costosa
- Si el atacante entra en el bastión, todavía tiene un router por delante (no puede hacer sniffing)



### **Detección de intrusos: IDS**

En muchas ocasiones, el atacante **enmascara** el ataque en tráfico permitido por el cortafuegos y por tanto para delatarlo se necesita un **IDS**. → (se usa cuando el atacante pasó el Firewall)

Son complementarios a los Firewalls.

#### Características deseables:

- Ejecución continua y autoanálisis
- Utilización de mínimos recursos posibles
- Adaptación a los cambios de sistemas y usuarios: **aprender por su experiencia y configurarse.**

#### **Tipos de IDS**

**NIDS: (NetworkIDS):** un IDS *basado en red*, detectando ataques a todo el segmento de la red. Su interfaz *debe funcionar en modo promiscuo* capturando así todo el tráfico de la red.

**HIDS:(HostIDS):** *basado en el host*, el principio de funcionamiento de un HIDS, *depende del éxito de los intrusos*, que generalmente dejan rastros de sus actividades en el equipo atacado, cuando intentan adueñarse del mismo, con propósito de llevar a cabo otras actividades. *El HIDS intenta detectar tales modificaciones en el equipo afectado, y hacer un reporte de sus conclusiones.*

→ **Hay sistemas pasivos (IDS) y sistemas reactivos (IPS)**

**Pasivos** → el sensor detecta una posible intrusión, almacena la información y manda una señal de alerta que se almacena en una base de datos.

**Reactivos** → Un sistema que reacciona ante el ataque previniendo que este continúe, se denomina **IPS** por sus siglas en inglés de "Intrusion prevention system". (el IDS responde a la actividad sospechosa reprogramando el cortafuegos para que bloquee tráfico que proviene de la red del atacante.)

### Jarrón de miel (HONEY POT)

En ocasiones es interesante *aprender de los propios atacantes*.

→ Para ello, en las redes se ubican **servidores puestos adrede** para que los intrusos los saboteen y son monitorizados por sistemas que actúan como **puentes** a los servidores, registrando de forma transparente los paquetes que acceden a dichos servidores.

- Detectado un ataque (por modificación de la estructura de archivos), se recompone la traza del atacante (secuencia de paquetes registrados en el monitor puente) y se pasa a un **análisis forense**.
- Este análisis forense concluye, en caso de detectar un nuevo ataque, en **una nueva regla de detección**.

## SEGURIDAD - ENCRIPCIÓN

### Clasificación de problemas de seguridad

Los problemas de seguridad de las redes pueden dividirse de forma general en cuatro áreas interrelacionadas:

1. El **secreto**, encargado de mantener la información fuera de las manos de usuarios no autorizados. *(el secreto es lo que estamos protegiendo de un tercero)*
2. La **validación de identificación**, encargada de determinar la identidad de la persona/computadora con la que se esta hablando.
3. El control de **integridad**, encargado de asegurar que el mensaje recibido fue el enviado por la otra parte y no un mensaje manipulado por un tercero. *(que un dato no haya sido modificado por un tercero)*





4. El **no repudio**, encargado de asegurar la “firma” de los mensajes, de igual forma que se firma en papel una petición de compra/venta entre empresas.

## Seguridad: Secretos - **Criptografía**

El **criptoanálisis** se encarga de descifrar los mensajes

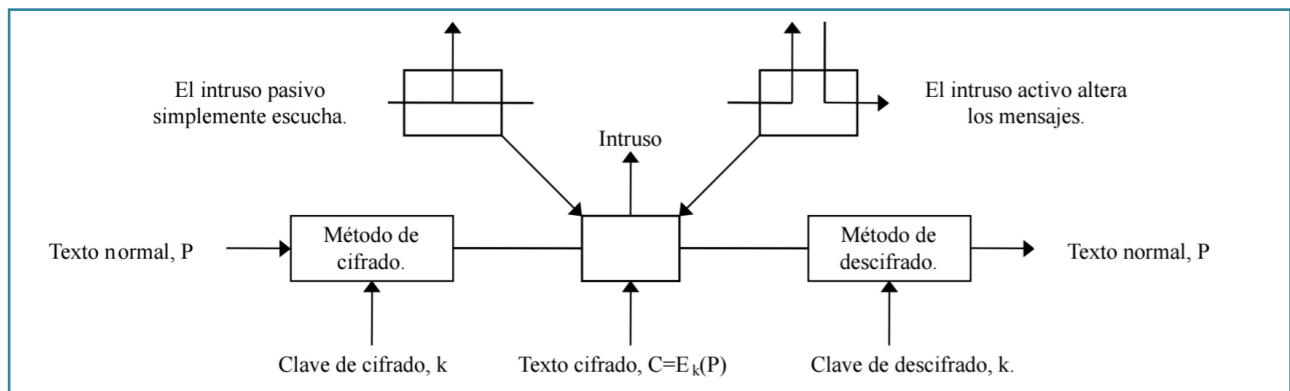
– Los intrusos utilizan estas técnicas.

La **criptografía** busca métodos más seguros de cifrado:

- **Criptografía clásica**: Algoritmo secreto. Cifrados por sustitución y transposición.
- **Criptografía moderna**: Algoritmo público, se conoce. Cifrado se basa en claves que se mantienen secretas.

## Cifrado: codificación de los mensajes

Con clave simétrica (misma clave  $k$ )



El **texto normal (P)** se transforma (cifra) mediante una función que tiene como parámetro una **clave  $k$**

**$C = E_k(P)$**  es el **texto cifrado (C)** obtenido a partir de **P**, usando la clave **k** y la **función matemática  $E_k$**  para codificar.

→  **$P = D_k(C)$**  es el descifrado de **C** para obtener el texto normal **P**.

## Ejemplos de cifrado

1. **Relleno de una sola vez**
2. **Criptografía clásica**
  - a. sustitución
  - b. transposición
3. **Criptografía moderna**
  - a. algoritmos de clave privada
  - b. algoritmos de clave pública

### 1. Relleno de una sola vez

Se escoge una **cadena como clave secreta**, por ejemplo “En un lugar d...”, y se va aplicando la función **XOR sobre el texto normal a cifrar, bit a bit.**

Texto normal o mensaje P = "texto cifrado"

Cadena de cifrado = "En un lugar de la Mancha de cuyo nombre..."

|                            |    |    |    |    |    |    |    |    |    |    |    |    |    |
|----------------------------|----|----|----|----|----|----|----|----|----|----|----|----|----|
| Texto original             | t  | e  | x  | t  | o  | c  | i  | f  | r  | a  | d  | o  |    |
| Codificación ASCII (hex)   | 74 | 65 | 78 | 74 | 6F | 20 | 63 | 69 | 66 | 72 | 61 | 6F |    |
| Texto de cifrado           | E  | n  |    | u  | n  |    | l  | u  | g  | a  | r  | d  |    |
| Codificación ASCII (hex)   | 45 | 6E | 20 | 75 | 6E | 20 | 6C | 75 | 67 | 61 | 72 | 20 | 64 |
| Codificación cifrada (hex) | 31 | 0B | 58 | 01 | 01 | 00 | 0F | 1C | 01 | 13 | 13 | 44 | 0B |

$$0x74 \text{ XOR } 0x45 = 0111\ 0100 \text{ XOR } 0100\ 0101 = 0011\ 0001 = 0x31$$

Para el descifrado, simplemente volvemos a aplicar con XOR la misma cadena de cifrado. → Tenemos que darle a la otra persona el *texto de cifrado*.

#### Inconvenientes:

- El **manejo de la clave** entre el emisor y el receptor (llevarla, o comunicarla, define el mensaje máximo que se puede transmitir)
- La **sincronización** entre ambos para que no se cometa ningún error en la transmisión
  - Si faltase un bit a partir de ahí todos los datos estarían alterados.

## 2. Criptografía Clásica

- a. **Cifrado por sustitución:** Este método **consiste en desplazar (a derecha o izquierda) el alfabeto de texto cifrado k letras**, siendo **k la clave de cifrado**. El descifrado es invertir el desplazamiento. El cifrado por sustitución más antiguo conocido es el del emperador Julio Cesar

Ejemplo: clave  $k=+3$ , texto normal P="ataque"

Texto normal:    abcdefghijklmnopqrstuvwxyz  
                    ↓     ↓                    ↓     ↓  
Clave:            defghijklmnopqrstuvwxyzabc

Texto cifrado C:    **dwdtxh**

→ **problema:** depende del tipo de texto, encuentras letras que se repiten y se descifra el código.

- b. **Cifrado por transposición:** Los cifrados por transposición **reordenan las letras (o los bits)**, mientras que los cifrados por sustitución conservan el orden de los símbolos de texto normal.

Ejemplo: texto normal P = "La clave secreta es AR", clave k = "cripto"

La clave k, se utiliza para reordenar columnas del texto normal (no debe tener letras duplicadas)

ordenación: "1-c 2-i 3-o 4-p 5-r 6-t"

Se coloca el texto normal

El texto cifrado se obtiene cogiendo el texto normal por columnas con la ordenación determinada por la clave

C = "LvR    tRacs csa aeeAlee "

cripto  
152463  
La cla  
ve sec  
retaes  
AR

**Problema transposición:** Conocido el contexto del mensaje y buscando las diferentes posibilidades de combinación de letras en palabras comunes, el criptoanalista puede determinar "fácilmente" la longitud de la clave y posteriormente la ordenación de columnas.

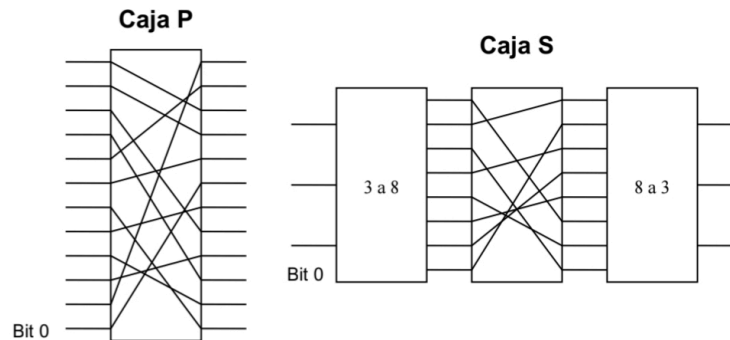
## ¿Transposición o sustitución?

Un texto está cifrado por transposición, cuando las frecuencias de las letras respecto al alfabeto se mantienen. Por ejemplo, la letra “a” seguirá apareciendo el 63% de las veces, mientras que la sustitución, si “a” se sustituye por “q”, entonces la letra “q” aparecerá el 63%.

## 2. Criptografía Moderna

Mismas ideas básicas que la criptografía tradicional, la transposición y la sustitución (que pueden implementarse mediante circuitos), pero con distinta orientación.

### Ejemplo Circuitos P y S



**Circuito P de permutación o transposición:** entrada/salida de 12 bits reordenados. Ejemplo 1010 1010 1010 (0xAAA) se convierte en 1110 0001 0011 (0xE13). **Se mantienen los mismos bits pero no el orden.**

**Circuito S de sustitución:** entrada/salida de 3 bits con decodificación/codificación (sustitución) interna. Ejemplo 000 se sustituye por 101. **Ni el orden ni los bits se mantienen.** Utilizan internamente una permutación.

→ **Cascada de circuitos P y S:** La potencia real de los circuitos P y S sólo se ve claramente cuando aparecen en cascada una serie completa de estas cajas para formar lo que se llama un cifrado de producto.

### Cifrado de clave privada y pública (1/2)

- En el **cifrado de clave privada (SIMÉTRICA)** las **claves de cifrado y descifrado son la misma** (o bien se deriva de forma directa una de la otra), debiendo mantenerse en **secreto** dicha clave.
  - **Es más rápido que el de clave pública** (de 100 a 1000 veces) =>
  - Se utiliza generalmente **en el intercambio de información una vez establecida una sesión**. Estas claves también son conocidas como claves de sesión o de cifrado simétricas, ya que en ambos extremos se posee la misma clave.
  - Ejemplos: – **DES** (Data Encryption Standard) y **T-DES** (o 3DES)– **IDEA** (International Data Encryption Algorithm)– **AES** (Advanced Encryption Standard) o Rijndael
- En el **cifrado de clave pública (ASIMÉTRICA)**, las **claves de cifrado y descifrado son independientes**, no derivándose una de la otra, por lo cual puede hacerse pública la clave de cifrado siempre que se mantenga en secreto la clave de descifrado.

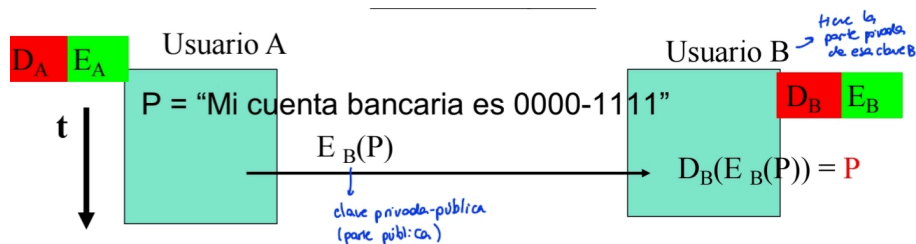
- Es más lento y por tanto **se utiliza para intercambiar las claves de sesión**. Como este algoritmo utiliza dos claves diferentes, una privada y otra pública el cifrado se conoce como **cifrado asimétrico**.
- Ejemplo:– Cifrado **RSA** (Rivest, Shamir, Adleman)



## Notación para cifrado y descifrado en clave simétrica y asimétrica

1. **Con claves simétricas**, cuando cifra el usuario A y utiliza su clave simétrica  $K_A$ :
  - $E_{K_A}(P)$  lo indicaremos como  $K_A(P)$
  - $D_{K_A}(P)$  lo indicaremos también por  $K_A^{-1}(P)$
2. **Con claves asimétricas**, cuando cifra el usuario A y utiliza su clave  $K_A$ , formada por parte privada y parte pública:
  - Con **parte pública** =  $E_{K_A}(P)$  lo indicaremos por  $E_A(P)$
  - Con **parte privada** =  $D_{K_A}(P)$  lo indicaremos por  $D_A(P)$

## Uso de clave pública para enviar algo de forma segura



A toma su primer mensaje  $P$ , calcula  $E_B(P)$  y lo envía a B

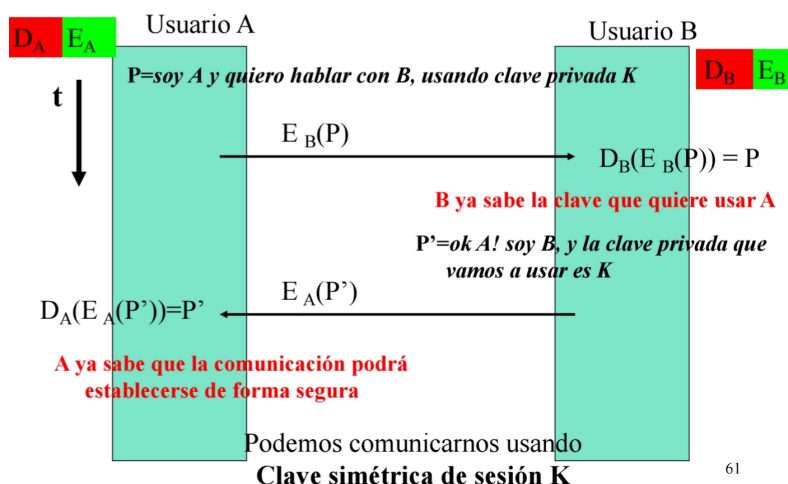
B entonces lo descifra aplicando su clave secreta  $D_B$ , es decir, calcula  $D_B(E_B(P)) = P$

Nadie más que B, puede descifrar el mensaje  $E_B(P)$ , porque se supone que el sistema de cifrado es robusto y porque es demasiado difícil derivar  $D_B$  de la  $E_B$  públicamente conocida

B procede igual que A.

60

## Uso de clave pública para intercambio de claves privadas



61